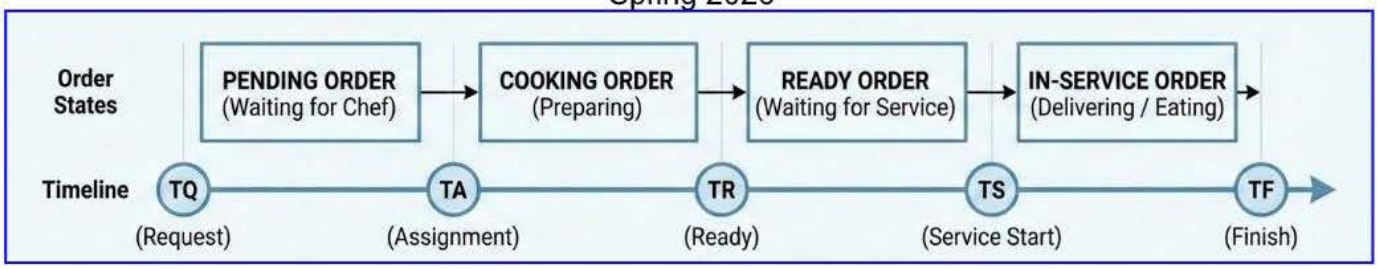
Data Structures and Algorithms

Restaurant Management

Project Requirements

Spring 2026



This project involves building a simulation of a restaurant's order management system. The system must handle both dine-in dining and delivery orders, manage resources (chefs, tables, scooters), and optimize for efficiency. The core challenge is to apply appropriate data structures to handle the dynamic and priority-based nature of restaurant operations.

Project Phase	*%*	*submission*
Phase 1.1 **Data structure Report**	**10%** **Team Grade**	**Sun, 15-Mar**
Phase 1.2	**30%** **Team Grade**	**Sat, 04-Apr**
Phase 2	**60%** **Indv + Team grade**	**Wed, 06-May**

Late Submission is not allowed.

The project code must be totally yours. The penalty of cheating (taking or giving) any part of the project from/to any other source is never tolerated is to have ZERO project grade in addition to other actions according to university regulations. It is totally your responsibility to keep your code private.

Project Documents

1. **Project Description (This document)**
2. **Project Phases and Deliverables**

Main Entities: Orders, Chefs, Scooters and Tables

Orders

The following should be loaded for each order from the input file:

- **Request timestep (T_Q):** When the order request was received.
- **Order Size:** the number of dishes for this order
- **Order Price:** the total order price the customer should pay.
- **Order Type:**
  - **Dine-in orders (OD)** should be served inside the restaurant (on its tables). There are two subtypes of dine-in orders
    - **dine-in Grilled (ODG)**: requiring specific cooking methods and should be prepared by special chefs.
    - **dine-in Normal (ODN)**: all other dine-in orders
  - **Takeaway orders (OT)** that are ordered on site and picked by the customer
  - **Delivery orders (OV)** should be delivered for away customers. There are three subtypes of delivery orders
    - **Delivery Cold (OVC)**: An order where some (or all) of it needs to be delivered cold
    - **Delivery Grilled (OVG)**: requiring specific cooking methods and should be prepared by special chefs.
    - **Delivery Normal (OVN)**: all other delivery orders
- **For dine-in orders only**
  - **No. of seats:** required number of seats to reserve a suitable table
  - **Order duration:** the period a table should be reserved for this order (in timesteps)
  - **Can Share?** Does the customer agree to share the table with others ?
- **For delivery orders only**
  - **Order distance:** in meters

chefs

The restaurant has a number of available chefs that should prepare the orders.
At startup, the system loads (from a file) information about the available **chefs**. For each chef, the system should load the following information:

- **Specialization:** There are 2 types
  - **Special chefs (CS)** who can prepare grilled orders
  - **Normal chefs (CN)** who can prepare other orders
- **Speed:** the number of dishes a chef can prepare in one timestep. The chef speed is the same for all chefs of the same type.

Scooters

Scooters are used to deliver orders. Scooter information to load:

- **Speed:** in meters/timestep.
- **Maintenance Duration (Main_Dur)**:The interval needed to fix/maintain a scooter. Each scooter should go to maintenance after being assigned to **Main_Ords** orders. (Main_Ords is loaded from the input file)

Tables

Tables are used to serve dine-in orders. Table information to load:

- **Capacity:** Total number of seats

Orders Assignment Criteria

After an order is received, it should encounter two assignment stages:

Stage1 - Assign to a chef to prepare then serve orders in stage2
Stage2 - Serve dine-in orders by **assigning to a table.** Deliver delivery orders by **assigning to a scooter**. Takeaway orders are picked by the customer once ready.

☞ In case a suitable chef/table/scooter is not available, the order should wait to the next timestep and check for a availability again

Stage1 Order-chef assignment Criteria:

1) First, assign OD orders in the following sequence:
    - ODG orders are assigned to a CS only
    - Then, ODN orders are assigned to a CN. If no CNs are available, assign to a CS.
2) Then assign OT orders to a CN only.
3) Then assign OV orders in the following sequence:
    - OVG orders are assigned to CS only
    - Then, OVC orders are assigned to a CN. If no CNs are available, assign to a CS.
    - Then, OVN orders are assigned to CN only
4) All orders are assigned to chefs based on the FCFS criteria except for OVG orders where they should be prioritized according to order price, size and distance.

☞For OVG orders, you should develop a reasonable **weighted** priority equation depending on, at least, the above factors

After stage1, the order is **ready** to be served/delivered

Stage2 Assignment Criteria:

1) Ready Takeaway orders should wait 1 timestep to pack then picked by the customer.
    - Customers pick orders based on FCFS criteria
2) Ready dine-in orders should be assigned to a suitable table.
    - Opt for table sharing first.
    - You should follow the best-fit criteria (fit the order to the table with minimum possible available seats)
3) Ready delivery orders should be assigned to a scooter based on FCFS criteria.
    - First, assign all OVC orders, then any other ready OV order.
    - Scooters should be assigned so that a scooter that cuts shorter distances so far should be assigned first.

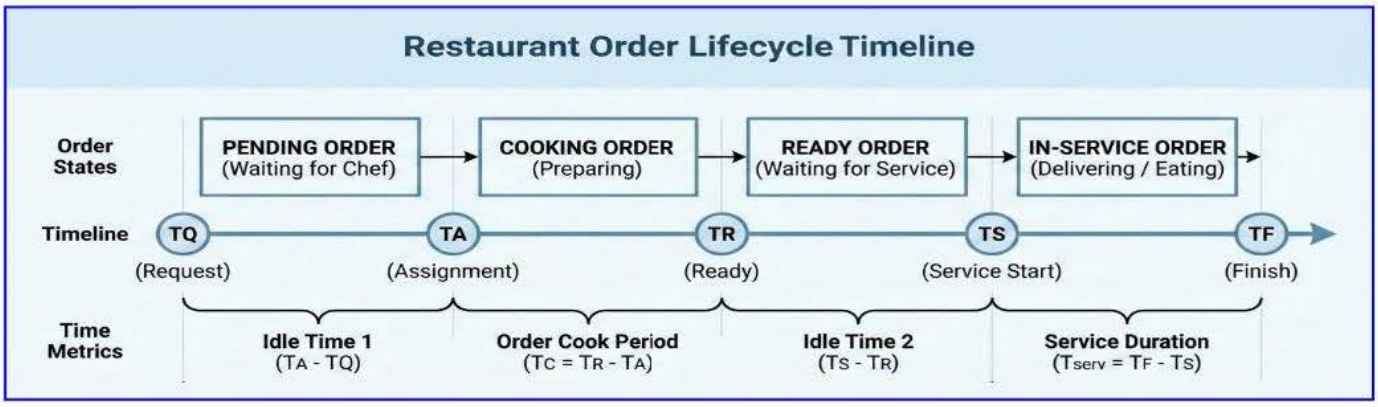
☞ If an order cannot be assigned at the current time step, it should wait for the next time step and so on.

There are some additional services the restaurant offers to customers:

- **Order cancellation for OVC orders only**
  When a cancellation request is received for an order,
    - ☐ If the order is ***pending*** or ***ready***, just drop it.
    - ☐ If it is ***cooking***, stop cooking immediately, remove the order and make its chef available

Simulation Approach & Assumptions

You will use incremental time steps. You will divide the time into discrete time steps of 1 unit time each and simulate the changes in the system at each ***timestep***.



Definitions

- **Order Request Time (T_Q):** The timestep when the order is issued by the customer.
- **Pending Order:** The order that has been requested (i.e. Order T_Q < current timestep) but not assigned to a chef yet.
- **Cooking Order:** that is being prepared (assigned to a chef) but not done yet.
- **Ready Order**: The order that is done by the chef but not served yet.
- **In-Service Order**: The order that is on a table but not done yet or on its way to be delivered.
- **Finished Order:** The order that is delivered or fully served.
- **Assignment Time (T_A):** the timestep an order is assigned to a chef to prepare.
- **Ready Time(T_R):** The timestep when the order becomes ready (done by a chef)
- **Service Start Time(T_S)**:The timestep when an order is assigned to a table/scooter
- **Order Idle Time (T_i):** $T_i = (T_A - T_Q) + (T_S - T_R)$
- **Order Cook Period (T_C):** Time interval to cook the order $T_c = T_R - T_A$
- **Customer Wait Time (T_w):** $T_w = T_i + T_c$
- **Service Duration (T_{serv}):** Time interval to deliver (for **OV**) or to release the table (for **OD**)
- **Order Finish Time (T_F):** The timestep when the order is finished completely (For OD: it means the order table is free again. For OV: it means the order has reached its destination, For OT: the order is picked by the customer)

Bonus (maximum 10% of project grade)

- **[3%] Order overwait for Ready OVG only**
  For a **ready** OVG order, if ***currentTime - T_R > TH***, it is considered as an overwait order. Overwait orders should be placed in a separate list. Orders in this list have the highest assignment priority. They should be assigned a scooter before any other orders. Overwait list should be arranged so that an order with higher ***currentTime - T_Q*** *value* should be assigned first. (Note: **TH** is the overwait threshold)

- **[3%] Rescue Scooter:** With a very low probability a scooter might fail to deliver an order due to mechanical breakdown. Upon failure, a rescue scooter must start to move to take over. Once the rescue scooter arrives at the place of the failed scooter, the failed scooter should start its journey back to the restaurant and the rescue scooter should complete the delivery from this point on. When a failed scooter arrives at the base stations, it must go to the maintenance list.

- **[4%] COMBO Orders:** These are orders that need to be prepared by up to 4 chefs (at least one of them is CS). They must be delivered by 2 or more scooters. You must update the input file to add requests for such orders with full details. These orders have the highest assignment priority in any stage over ALL other orders.

To implement the bonus parts, you should

- If needed, update the DS or add a new DS.
- Modify the input file to add more information to be uploaded as appropriate
- Modify the output screen to display info about the newly added lists.
- Add more info to the output file per mission and more statistics.

Input/Output File Formats

Your program should receive all information to be simulated from an input file and produce an output file that contains some information and statistics about the simulation.

The Input File line-by-line Format

- **CN CS** (the number of chefs of different types)
- **CN_speed CS_speed** (dishes/timestep)
- **S_count S_speed** (the number of scooters and speed)
- **Main_Ords Main_Dur**
- **Total_no_of_Tables**
- **Table_count Capacity (**repeated pairs to cover all tables**)**
- **TH** (overwait threshold)
- The next line contains one integer ***M*** that represents the number of **orders actions**
- Then the input file contains ***M*** lines (one line for **each action**):An action may be:
  - ❑ Order Request action. Denoted by letter **Q**, or
  - ❑ Cancellation action for an existing order. Denoted by letter **X**.

NOTE: The lines of all actions are **sorted by the Request time** (**T_Q**) in **ascending** order.

Actions Lines

- **Request Action Line** has the following info
  - ☐ **Q**(letter Q in the beginning of its line) means a reQuest action
  - ☐ **TYP** is the order type (O*DG, ODN, OT, OVC,...etc)*
  - ☐ **T_Q** is the request timestep
  - ☐ **ID** is a unique sequence number that identifies each order.
  - ☐ **SIZE** is the number of dishes of the order
  - ☐ **Price** is the order price.
  - ☐ **No. of Seats**: for OD orders only
  - ☐ **Order Duration**: for OD orders only
  - ☐ **CanShare**: for OD orders only
  - ☐ **Distance**: for OV only (in meters)
- **Cancellation Action line** has the following info
  - ☐ **X**(Letter X) means an order cancellation action
  - ☐ **Tcancel** is the action timestep. (Cancellation timestep)
  - ☐ **ID** is the id of the order to be canceled. This ID must be for an OV order.

Sample Input File

25	**13**									no. of chefs for each type
3	**2**									chefs speeds for each type
20	**150**									no. of scooters & scooter speed(meters/timestep)
8	**23**									Scootes maintainance info line
20										Total no. of tables
										⇒ 5 tables with 5 seats, 2 with 4 seats, 7 with 5 seats,....
800										no. of request actions in this file
Q	**ODG**	**7**	**1**	**15**	**110**	**4**	**14**	**Y**		Request action example
Q	**OVC**	**9**	**2**	**7**	**56**	**2**	**150**			
Q	**ODN**	**9**	**3**	**21**	**300**	**8**	**30**	**N**		
X	**15**	**2**								Cancellation action example
... and so on for 800 requests										

The Output File Format (Phase 2)

The output lines should be as follows for each FINISHED order

T_F **ID** T_Q T_A T_R T_S T_i T_C T_w T_{serv}

The output lines **must be sorted** by finish time T_F in **descending** order.

Then the following statistics should be shown at the end of the file with descriptive messages

1- Total number of orders and total number of each order type
2- Total number of chefs and total number of each type
3- Total number of scooters and total number of each type
4- Percentage of finished orders and percentage of cancelled orders
5- Percentage of overwait orders
6- Average for T_i T_C T_w T_{serv} for all finished orders
7- **Scooters utilization % (Busy means not waiting at the restaurant)**
8- **Chefs utilization % (Busy means preparing an order)**

Program Interface

The program can run in one of two modes: **interactive,** or **silent mode**. When the program runs, it should ask the user to select the program mode. **Interactive mode**: allows users to monitor the restaurant operation.

```
Current Timestep:78
=============== Actions List ================
For reQuest action:print [Order Type, TQ, order ID], For cancel print (X, Tcancel, order ID)
512 actions remaining: [ODG , 100, 333], (X, 105, 79) ….. and so on
—> Print ONLY the first 10 actions currently in the actions list

————————— Pending Orders IDs ———————————
For each pending list print
List count, order type, IDs of all orders in the list
————————— Available chefs IDs ———————————
2 CS : 10, 22
10 CN : 2,7, ….. (print all)
```

```
————————————    Cooking orders [Orders ID, chef ID] ———————————————————
23 cooking orders: [55, 3], …… (print all)
————————————    Ready Orders IDs ———————————————————
For each Ready list print
List count, order type, IDs of all orders in the list
————————————   Available scooters IDs ———————————————————
8 Scooters : 17, 12, 33, ….. (print all)
————————————   Available tables [ID, capacity, free seats] ———————————————————
30 tables : [T7, 8, 3], ….. (print all)

————————————   In-Service orders [order ID, scooter/Table ID]  ———————————————————
15 Orders: [75, S14], [6, T3] ….. (print all)

————————————   In-maintainance scooters IDs   ---------------------
3 scooters: 40, 13, 9

————————————    Scooters Back to Restaurant  IDs   ---------------------
2 scooters: 74, 93

————————————   Cancelled Orders IDs   ---------------------
1 cancelled: 68
————————————   Finished orders IDs----------------------------
54 Orders: 39, 2,  19, 1 ……(print IDs of all finished orders in descending order of finish time )
PRESS ANY KEY TO MOVE TO NEXT STEP !
```

In **silent mode**, the program produces only an output file (See the "File Formats" section). It does not print simulation details on the screen. It just prints:

```
Simulation Starts in Silent mode ...
Simulation ends, Output file created
```

No matter which interface mode is used, the output file MUST be always created